Homework 2 — Image Classification

1. Environment

The homework runs on Google Colab. Copy the package folder to Google Drive, open `HW2.ipynb` from it in Colab, and select a GPU runtime.

```
python >= 3.10
torch >= 2.3, torchvision >= 0.18, timm >= 1.0, numpy, matplotlib
```

Every script runs from the package root. CIFAR-10 (170 MB) is downloaded on first use; results go to `results/` and checkpoints to `checkpoints/`.

```
python check.py                          # self-check, see section 4
python p1_convnet.py --recipe none
python p1_convnet.py --recipe full
python p2_modern_convnet.py
python p3_vit.py
python p4_pretrained_vit.py
```

2. What you implement

Problem	Functions	Points
P1 ConvNet	none: run and analyse	10
P2 ModernConvNet	`FeedForward`, `ModernConvBlock`, `ModernConvNet` in `models/modern_convnet.py`	25
P3 ViT	`PatchEmbedding`, `Attention.forward` in `models/vit.py`	25
P4 Pretrained ViT	`extract_features`, `train_linear_probe`, `configure_finetune` in `p4_pretrained_vit.py`	15
P5 Written questions	none	25

Each place to fill is marked with `TODO: implement this` followed by `raise NotImplementedError`. Delete both lines and write your code there. Keep the function signatures and the attribute names given in the docstrings; the self-check and the grading load weights through them. Do not change any other file.

The questions are in `report_template.docx`.

3. Rules

You may use `torch`, `torch.nn` layers and `torch.nn.functional`, except for the items below, and `timm` only through `load_backbone` in `p4_pretrained_vit.py`.

You may not use, in `models/` or in the functions you write in `p4_pretrained_vit.py`:

`torch.nn.functional.scaled_dot_product_attention`, `torch.nn.MultiheadAttention`, `torch.nn.TransformerEncoderLayer`, `torch.nn.Unfold`, `torch.nn.functional.unfold`, and any layer or model from `timm` or `torchvision`.

`PatchEmbedding` must not use `nn.Conv2d`.

4. Self-check

```
python check.py
```

The script tests each function you write on small random inputs and checks the size of each network. It runs on the CPU in a few seconds and needs neither CIFAR-10 nor the pretrained model, so run it before you spend GPU time. It reports `SKIP` for anything not implemented yet.

`check.py` is a debugging aid, **not the grader**.

- Passing every check does not mean full marks: the trained results and the written answers count for as much as the code.
- Failing a check does not mean zero. If you made a different but defensible choice, say so in the report and you keep most of the credit.

Paste the full output into section 2 of your report.

5. Fixed settings

Every network trained from scratch uses the same recipe, set in `engines.py`. Do not change it.

Setting	Value
Optimizer	AdamW, learning rate 1e-3, weight decay 0.05
Batch size, epochs	256, 120
Learning rate schedule	linear warm-up from 0 over 3 epochs, then cosine decay to 0
Mixed precision	float16 autocast on the GPU
Seed	0

Recipe	Augmentation	Label smoothing	Stochastic depth
`full` (P1 to P3)	random crop (padding 4), horizontal flip, RandAugment (2 ops, magnitude 9), random erasing (p = 0.25), MixUp (alpha 0.8) or CutMix (alpha 1.0)	0.1	0 to 0.1
`none` (P1 only)	none	0	0

Network	Parameters	MACs per image
ConvNet	4.81 M	241.0 M
ModernConvNet	4.74 M	244.1 M
ViT	4.77 M	315.4 M

The scripts evaluate on the 10,000 CIFAR-10 test images after every epoch; there is no separate validation split. Under the `full` recipe, training accuracy is measured on the augmented and mixed images against their original labels.

Conventions.

- `PatchEmbedding` orders the tokens row by row, left to right, and flattens each patch in (row, column, channel) order.
- `Attention` splits the channels into heads in order: head h takes channels h x head_dim to (h + 1) x head_dim - 1.
- The DropPath rate of block i is given by `drop_path_rates`, rising from 0 in the first block to the maximum in the last.

6. Running on Colab

Run one problem per Colab session. A training script writes a checkpoint after every epoch; started again with the same arguments, it continues from the last completed epoch, and a finished run only rewrites its result files. `--restart` discards the checkpoint. Keep the package folder in Google Drive so that checkpoints and results survive the end of a session.

Each run writes `results/<run>.json`, with the full learning curves, and `results/<run>_curves.png`. Problems 1 to 3 also write a comparison figure.

7. Submission

Submit a single archive named `HW2_<student id>_<name>.zip` containing:

```
HW2_<student id>_<name>/
├── models/                your implementation
├── p4_pretrained_vit.py   your implementation
├── results/               every .json and .png the scripts wrote
└── report.pdf             at most 10 pages
```

Do not include `data/` or `checkpoints/`.